

CLASE 15: EXAMEN FINAL — PROGRAMACIÓN (JAVA POO)

Información general

Bootcamp: Programación Cero a POO

Clase: 15 — Examen Final

Duración: 90 minutos

Modalidad: Individual

Lenguaje: Java

Entrega: Repositorio GitHub (commits obligatorios)

Objetivo del examen

Evaluar la capacidad del estudiante para modelar un problema usando Programación Orientada a Objetos, aplicando clases, objetos, encapsulamiento, herencia, polimorfismo, colecciones, validaciones y manejo básico de errores.

Caso de estudio

Sistema de gestión de mascotas

Se desea desarrollar una aplicación en consola que permita gestionar diferentes tipos de mascotas.

El sistema debe permitir crear mascotas, almacenarlas en una lista, mostrarlas y ejecutar acciones propias de cada tipo de mascota.

Todas las mascotas comparten características generales, pero cada tipo tiene un comportamiento diferente.

Requerimientos

1. Clase padre: Mascota

Debe representar la información general de cualquier mascota.

Atributos mínimos:

- id (int)
- nombre (String)

- edad (int)

Métodos mínimos:

- Constructor
- Getters y setters
- Método:

```
public void mostrarInfo()
```

Este método debe mostrar los datos de la mascota.

- Método:

```
public String hacerSonido()
```

Este método representa el comportamiento de la mascota.

En la clase padre puede retornar un mensaje general.

2. Herencia (OBLIGATORIO)

Se deben crear al menos dos subclases:

- Perro
- Gato

(Puede agregar otra si desea, por ejemplo: Ave)

Cada subclase debe:

- Heredar de Mascota usando extends
- Tener su propio constructor
- Sobrescribir el método hacerSonido()

Ejemplo esperado:

- El perro puede decir "Guau guau"
 - El gato puede decir "Miau"
 - El ave puede decir "Pío pío"
-

3. Polimorfismo (OBLIGATORIO)

Se debe utilizar una lista:

`ArrayList<Mascota>`

Importante:

Aunque existan objetos Perro o Gato, deben almacenarse como tipo Mascota.

Al recorrer la lista y llamar al método `hacerSonido()`, cada objeto debe responder según su tipo.

4. Menú del sistema

El programa debe implementar un menú repetitivo con las siguientes opciones:

Opción 1: Registrar mascota

El usuario debe elegir el tipo de mascota:

- Perro
- Gato
- Ave (opcional)

Luego ingresar:

- id
- nombre
- edad

Validaciones:

- `id > 0`
- nombre no vacío
- `edad > 0`

Si hay error, mostrar mensaje y no registrar.

Opción 2: Mostrar mascotas

Mostrar todas las mascotas registradas:

- id
 - nombre
 - edad
 - tipo de mascota
 - sonido que realiza (usar hacerSonido())
-

Opción 3: Ejecutar sonido de una mascota

Buscar una mascota por id.

Si existe, mostrar el resultado del método hacerSonido().

Si no existe, mostrar: "Mascota no encontrada".

Opción 4: Buscar mascota por nombre

Buscar una mascota por su nombre.

Si existe, mostrar toda su información.

Si no existe, mostrar: "Mascota no encontrada".

Opción 5: Salir

Manejo de errores

Se debe utilizar al menos un bloque try-catch para evitar errores de entrada.

Consideraciones importantes

- No hacer todo en una sola clase
- Usar encapsulamiento (atributos privados)
- Aplicar herencia correctamente

- Aplicar polimorfismo al recorrer la lista
 - Validar entradas
 - Mantener código ordenado
-

Casos de prueba obligatorios

- id igual a 0
 - edad negativa
 - nombre vacío
 - búsqueda inexistente
 - lista vacía
 - diferentes tipos de mascotas
-

Entrega

Repositorio GitHub obligatorio

Evidencia mínima:

- Código funcional
- Al menos 3 commits

Ejemplo:

- feat: estructura inicial
- feat: clases Mascota, Perro y Gato
- feat: menú y funcionalidades

No es necesario README.

Criterios de evaluación (sobre 10)

- Modelado de clases: 3 puntos
- Herencia y polimorfismo: 3 puntos

- Funcionalidad: 2 puntos
 - Validaciones: 1 punto
 - Organización: 1 punto
-

Recomendación

Antes de programar:

- Definir la clase padre
- Definir las subclases
- Pensar qué comportamiento cambia
- Luego implementar